GPU, AI infrastructure and LLM serving

Know what limits inference on a GPU, run vLLM, TGI and Triton, schedule GPUs on Kubernetes, and control latency and cost.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. GPU basics: memory and what limits inference (15 min)
2. Serving stacks: vLLM, TGI and Triton (14 min)
3. GPUs on Kubernetes (14 min)
4. Scaling, latency and cost (14 min)
5. Monitoring and debugging GPU workloads (15 min)
6. Quantization and sizing a model for a GPU (12 min)
7. Batching, KV cache and throughput (13 min)
8. Multi-GPU serving: tensor and pipeline parallelism (12 min)
9. Autoscaling LLM inference (13 min)
10. Model loading, caching and cold starts (12 min)
11. GPU sharing: MIG, time-slicing and MPS (12 min)
12. GPU failures, Xid errors and recovery (12 min)
Cheat sheet

Lesson 1. GPU basics: memory and what limits inference

15 minutes. Work out how much GPU memory a model needs, and why generating text is limited by memory bandwidth more than by compute.

A large language model is mostly a very big list of numbers that must be read for every word it produces. Whether it runs, how fast, and for how many users depends on a few simple sums. This lesson gives you the sums.

What you will learn

- What a GPU is good at and what its memory looks like
- How to estimate memory for weights and the KV cache
- Prefill versus decode, and why decode is bound by memory bandwidth
- Quantization and what it trades

What a GPU offers

A GPU has thousands of simple cores that do many multiplications at once, which suits the matrix arithmetic in neural networks. It has its own fast memory called **VRAM** (or HBM on data center cards). A model must fit in VRAM to run well. Data centre cards range from 24 GB to 80 GB or more per device, and a model that does not fit must be split across several cards or compressed.

Weights

Memory for the model's **weights** is the parameter count times the bytes per parameter.

Precision	Bytes per parameter	8B model	70B model
FP16 or BF16	2	about 16 GB	about 140 GB
INT8 / FP8	1	about 8 GB	about 70 GB
4-bit	about 0.5	about 4 to 5 GB	about 35 to 40 GB

A 70B model in 16-bit needs more than one 80 GB card. In 4-bit it fits on one, with some loss of quality that depends on the method and model.

The KV cache

While generating, the model keeps the **key and value** vectors of every token seen so far, so it does not recompute them. This is the **KV cache**, and it grows with each token and each concurrent request.

Bytes per token = 2 (key and value) × layers × KV heads × head size × bytes per value.

For a model with 32 layers, 8 KV heads (grouped-query attention) and head size 128 in 16-bit:

2 × 32 × 8 × 128 × 2 = 131 072 bytes, which is 128 KiB per token.

A 4 000-token conversation takes about 0.5 GB, and 64 such conversations take about 32 GB. That is why serving many users needs far more memory than the weights alone, and why memory runs out before compute does.

Prefill and decode

Generating a reply has two phases.

- **Prefill** reads the whole prompt at once. It is compute-heavy and fast per token, and it sets the **time to first token (TTFT)**.
- **Decode** produces one token at a time. For each token, the GPU must read all the weights from memory. The speed is therefore close to memory bandwidth divided by model size.

MARGA by EknathaLabs marga.eknathalabs.com

A rough ceiling for one stream: a card with 2 000 GB/s of bandwidth and a 16 GB model can read the weights about 125 times per second, so about 125 tokens per second for one user, in theory. In practice you get less. **Batching** serves many users with the same read of the weights, which is how throughput rises. That is why batch size and memory for the cache decide cost per token.

Quantization

Quantization stores weights, and sometimes the KV cache, with fewer bits. It reduces memory and increases speed, because less data is read per token. Check quality on your own tasks before using an aggressive setting: small models and tasks needing exact numbers are more sensitive.

```
$ nvidia-smi --query-gpu=name,memory.total,memory.used --format=csv
name, memory.total [MiB], memory.used [MiB]
NVIDIA A100-SXM4-80GB, 81920 MiB, 40213 MiB
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GPU basics: memory and what limits inference fits together

Part	What it does	What goes wrong
What a GPU offers	A GPU has thousands of simple cores that do many multiplications at once, which suits the matrix arithmetic in neural networks.	Assuming more GPU cores will fix slow decoding when bandwidth is the limit.
Weights	Memory for the model's weights is the parameter count times the bytes per parameter.	Using aggressive quantization with no quality test.
The KV cache	While generating, the model keeps the key and value vectors of every token seen so far, so it does not recompute them.	Testing with one request and shipping.
Prefill and decode	Generating a reply has two phases.	Sizing a GPU by weights only.
Quantization	Quantization stores weights, and sometimes the KV cache, with fewer bits.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What a GPU offers to Weights / Weights to The KV cache / The KV cache to Prefill and decode / Prefill and decode to Quantization

Incident: The model that loaded and then crashed

Incident: *A 13B model loaded fine on a 24 GB card and crashed under the first real traffic.*

```
$ nvidia-smi --query-gpu=memory.used,memory.total --format=csv,noheader
23410 MiB, 24576 MiB
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 1.20 GiB.
GPU 0 has a total capacity of 24.00 GiB of which 0.81 GiB is free.
```

The weights in 16-bit took about 26 GB, so the team had loaded an 8-bit version of about 13 GB, which left about 10 GB. That looked like plenty in testing with one request. Under real traffic the KV cache of many long prompts used all of it. They reduced the maximum context length, limited concurrent requests, and moved to a larger card. Memory has to be planned for weights plus cache at peak load, not at the first test.

Common mistakes

- Sizing a GPU by weights only
- Assuming more GPU cores will fix slow decoding when bandwidth is the limit
- Using aggressive quantization with no quality test
- Testing with one request and shipping

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

About how much memory do the weights of a 70B model need in 16-bit?

Answer: About 140 GB

What limits the speed of decoding one token at a time?

Answer: Memory bandwidth, because all weights are read for each token

What does the KV cache store?

Answer: Key and value vectors of tokens already processed

Why is a quality test needed before using 4-bit quantization?

Answer: Compression can lose accuracy on some tasks

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Serving stacks: vLLM, TGI and Triton

14 minutes. Compare the main LLM serving tools, understand continuous batching and paged attention, and read the flags that matter.

Running a model in a notebook is easy. Serving it to many people at once, with steady latency and good use of the GPU, needs a serving engine.

What you will learn

- Why a plain loop wastes the GPU
- Continuous batching and PagedAttention
- The main engines: vLLM, TGI, TensorRT-LLM and Triton
- The vLLM flags you will tune

Why a loop is not enough

A simple server handles one request, or a fixed batch of requests, at a time. Replies have different lengths, so a fixed batch waits for the longest one while the GPU idles for the finished ones. New requests wait until the batch ends.

Continuous batching (also called in-flight batching) fixes this. After every generated token, the engine can drop finished sequences and add waiting ones, so the GPU stays full and new users start sooner.

PagedAttention, used by vLLM, stores the KV cache in small fixed pages instead of one large block per request. This reduces wasted memory from fragmentation, so more requests fit in the same GPU. **Prefix caching** reuses the cache for a shared prompt start, such as a long system prompt.

The engines

- **vLLM**: an open-source engine with continuous batching, PagedAttention and an OpenAI-compatible HTTP API. A popular default for open models.
- **Text Generation Inference (TGI)**: Hugging Face's server with similar features, tied to the Hugging Face ecosystem.
- **TensorRT-LLM**: NVIDIA's library that compiles models into optimised kernels for NVIDIA GPUs. It can be the fastest on those cards, with more setup work.
- **Triton Inference Server**: NVIDIA's general serving platform for many model types, which can host TensorRT-LLM and other backends.
- **Hosted APIs**: the provider runs the stack, and you pay per token.

Choose by model support, hardware, operational skill and how much you want to tune. Many teams start with vLLM and measure.

A vLLM example

```
$ vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --max-model-len 8192 \
    --gpu-memory-utilization 0.90 \
    --max-num-seqs 64
INFO  Loading model weights took 14.99 GiB
INFO  GPU KV cache size: 41,216 tokens
INFO  Maximum concurrency for 8192 tokens per request: 5.03x
INFO  Uvicorn running on http://0.0.0.0:8000
$ curl -s localhost:8000/v1/chat/completions -H 'Content-Type: application/json' \
    -d '{"model":"meta-llama/Llama-3.1-8B-Instruct","messages":[{"role":"user","content":"hello"}] \
    ,"max_tokens":20}'
```

Key settings:

- `--max-model-len`: the longest context accepted. Setting it higher makes the engine reserve for longer requests and reduces how many fit.

MARGA by EknathaLabs marga.eknathalabs.com

- `--gpu-memory-utilization`: the fraction of VRAM the engine may use, including the cache. If other processes share the GPU, lower it.
- `--max-num-seqs`: how many sequences run together.
- `--tensor-parallel-size N`: split each layer across N GPUs, needed when the model is larger than one card. It needs fast links between GPUs.

The log line "GPU KV cache size" is the most useful number at startup. It says how many tokens of cache the memory can hold, and so how many concurrent users with how long a context.

Operating notes

- Model weights are large (tens of GB). Cache them on a persistent volume so a restart does not download them again.
- Start-up takes minutes. Use readiness probes, and do not send traffic until the model is loaded.
- Pin the engine version, and test an upgrade, because behaviour and defaults change.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Serving stacks: vLLM, TGI and Triton fits together

Part	What it does	What goes wrong
Why a loop is not enough	A simple server handles one request, or a fixed batch of requests, at a time.	Setting a very large max-model-len that nobody uses.
The engines	vLLM: an open-source engine with continuous batching, PagedAttention and an OpenAI-compatible HTTP API.	Comparing engines with prompts that are nothing like real traffic.
A vLLM example	Key settings.	Running two engines on one GPU without lowering memory utilization.
Operating notes	Model weights are large (tens of GB). Cache them on a persistent volume so a restart does not download them again.	Redownloading weights on every restart.

Flow: Why a loop is not enough to The engines / The engines to A vLLM example / A vLLM example to Operating notes

Incident: The server that was slow with one GPU idle

Incident: *A two-GPU server delivered only the speed of one.*

```
$ nvidia-smi --query-gpu=index,utilization.gpu,memory.used --format=csv,noheader
0, 98 %, 76210 MiB
1, 0 %, 4 MiB
$ ps aux | grep vllm | head -1
vllm serve mistralai/Mixtral-8x7B-Instruct-v0.1 --max-model-len 16384
```

The engine had been started without `--tensor-parallel-size 2`, so it used only the first GPU, squeezed the model and the cache into it, and left the second card empty. After the flag was added the model was split over both cards, the cache became larger and throughput roughly doubled. The monitoring that revealed it was the per-GPU utilization, which had not been on the dashboard.

Common mistakes

- Setting a very large max-model-len that nobody uses
- Running two engines on one GPU without lowering memory utilization
- Redownloading weights on every restart
- Comparing engines with prompts that are nothing like real traffic

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does continuous batching do?

Answer: Adds and removes requests after every token so the GPU stays full

What does PagedAttention improve?

Answer: KV cache memory use, by avoiding fragmentation

What does --tensor-parallel-size 2 do in vLLM?

Answer: Splits each layer across two GPUs

Which log line tells you how much cache the server has?

Answer: GPU KV cache size in tokens

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. GPUs on Kubernetes

14 minutes. Schedule GPU workloads with the device plugin, understand why GPUs are whole units, and read the Pending-pod messages.

Kubernetes does not know what a GPU is until you tell it. Once you do, GPUs are scheduled like other resources, with a few differences that cause most of the trouble.

What you will learn

- The NVIDIA device plugin and GPU Operator
- Requesting GPUs, and why they are whole numbers
- Taints, node pools and cost
- Sharing GPUs with MIG and time-slicing

How GPUs reach pods

A GPU node needs three things: the NVIDIA driver, a container runtime hook that exposes devices to containers, and the **device plugin**, a daemonset that tells the kubelet how many GPUs the node has. The **NVIDIA GPU Operator** installs and manages these together. When it works, the node reports an extra resource:

```
$ kubectl describe node gpu-node-1 | grep -A2 'nvidia.com/gpu'
  nvidia.com/gpu:     2
...
  nvidia.com/gpu     0           0
```

A pod asks for GPUs in its container limits:

```
resources:
  limits:
    nvidia.com/gpu: 1
```

Extended resources such as this one must be set in `limits` (the request, if given, must equal it), and they are **integers**: you cannot ask for half a GPU, and they are not overcommitted. One pod holds its GPU even if it is idle.

Keeping other pods away

GPU nodes are expensive. Taint them so that ordinary pods do not land there and keep the node busy:

```
kubectl taint nodes gpu-node-1 nvidia.com/gpu=present:NoSchedule
```

GPU pods then need a matching **toleration**. A **node pool** or node group of GPU machines that scales from zero lets you pay only while jobs run, with some start-up delay.

When a pod stays Pending

```
$ kubectl describe pod llm-0 | tail -4
Events:
  Warning  FailedScheduling  default-scheduler  0/6 nodes are available: 2 Insufficient nvidia.com \
    /gpu, 4 node(s) had untolerated taint {nvidia.com/gpu: present}.
```

Read this as: two nodes have GPUs but all are in use, and the other four have a taint the pod does not tolerate. The fix is to free a GPU, add capacity, or fix the toleration, depending on which line matches your intent. If no node shows `nvidia.com/gpu` in its capacity at all, the device plugin or driver is not working: check the daemonset pods and the node logs.

Sharing a GPU

A whole GPU for a small model wastes money. Options:

- **MIG (Multi-Instance GPU)**: on some NVIDIA data centre cards, one GPU is split into isolated slices with their own memory and compute. Pods request a slice, such as `nvidia.com/mig-1g.10gb`.

MARGA by EknathaLabs marga.eknathalabs.com

- **Time-slicing**: several pods take turns on one GPU. There is no memory isolation, so one pod can exhaust memory and crash others, and latency becomes uneven.
- **MPS** and other sharing modes exist for specific cases.

MIG gives predictable performance and isolation. Time-slicing is for development and light, tolerant workloads.

Multi-GPU and multi-node

A large model needs several GPUs on **the same node** where possible, since links between GPUs inside a node (NVLink) are much faster than the network. A pod that asks for 4 GPUs needs a node with 4 free ones, and a cluster full of 2-GPU nodes can never schedule it. Distributed training across nodes needs fast networking and a scheduler that starts all pieces together (gang scheduling), or the job can deadlock with half its pods running.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GPUs on Kubernetes fits together

Part	What it does	What goes wrong
How GPUs reach pods	A GPU node needs three things: the NVIDIA driver, a container runtime hook that exposes devices to containers, and the device plugin, a daemonset that tells the kubelet...	Forgetting that drivers and the operator version must match the node image.
Keeping other pods away	GPU nodes are expensive.	Leaving GPU nodes untainted so system pods consume capacity.
When a pod stays Pending	Read this as: two nodes have GPUs but all are in use, and the other four have a taint the pod does not tolerate.	Requesting GPUs only in requests, or asking for fractions.
Sharing a GPU	A whole GPU for a small model wastes money.	Using time-slicing for production with strict latency.
Multi-GPU and multi-node	A large model needs several GPUs on the same node where possible, since links between GPUs inside a node (NVLink) are much faster than the network.	A pod that asks for 4 GPUs needs a node with 4 free ones, and a cluster full of 2-GPU nodes can never schedule it.

Flow: How GPUs reach pods to Keeping other pods away / Keeping other pods away to When a pod stays Pending / When a pod stays Pending to Sharing a GPU / Sharing a GPU to Multi-GPU and multi-node

Incident: Eight GPUs and a Pending pod

Incident: *A team had eight free GPUs in the cluster and their four-GPU job stayed Pending.*

```
$ kubectl get nodes -l accelerator=nvidia -o custom-columns=NAME:.metadata.name,GPU:.status.alloca \
    table.nvidia\.com/gpu
NAME          GPU
gpu-node-1    2
gpu-node-2    2
gpu-node-3    2
gpu-node-4    2
$ kubectl describe pod trainer-0 | grep FailedScheduling
0/4 nodes are available: 4 Insufficient nvidia.com/gpu.
```

The cluster did have eight free GPUs, but spread across four nodes with two each, and a pod must fit on one node. The team created a node pool of 8-GPU machines for large jobs and kept the 2-GPU nodes for small ones. The aggregate number of free GPUs was not the number that mattered.

Common mistakes

- Leaving GPU nodes untainted so system pods consume capacity

MARGA by EknathaLabs marga.eknathalabs.com

- Requesting GPUs only in requests, or asking for fractions
- Using time-slicing for production with strict latency
- Forgetting that drivers and the operator version must match the node image

Check yourself

How must a pod request a GPU?

Answer: In container limits as nvidia.com/gpu with a whole number

Why taint GPU nodes?

Answer: So ordinary pods do not occupy expensive GPUs

What is MIG?

Answer: Splitting a GPU into isolated slices with their own memory and compute

Eight GPUs are free in total across four 2-GPU nodes. Why can a 4-GPU pod not run?

Answer: A pod must fit on one node

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Scaling, latency and cost

14 minutes. Measure what users feel (TTFT and token speed), scale on the right signal, and work out the cost of a token.

GPU inference costs money every minute it runs, whether or not anyone is using it. Good operation means knowing what the users experience, scaling to match demand, and being able to say what a million tokens cost.

What you will learn

- The latency numbers: TTFT, inter-token latency and total time
- Why CPU-based autoscaling does not work for GPUs
- Scaling on queue depth and concurrency
- Cost per million tokens and how utilisation changes it

What users feel

- **TTFT (time to first token)**: how long before anything appears. It includes queue wait and prefill. Long prompts and busy servers raise it.
- **Inter-token latency (ITL)**: the gap between tokens after the first. At about 20 tokens per second per user the text keeps ahead of reading speed.
- **Throughput**: tokens per second summed across users, which sets cost.
- **p95 and p99**: slow outliers matter more than the average. Track percentiles.

Latency and throughput pull against each other. A bigger batch raises total throughput and slows each user a little. Set a target, such as "p95 TTFT under 1.5 seconds and 20 tokens/s per user", and let it decide the batch limits.

```
$ curl -s localhost:8000/metrics | grep -E 'vllm:(num_requests_(running|waiting)|gpu_cache_usage_p \
    erc)'
vllm:num_requests_running{model_name="llama-8b"} 64
vllm:num_requests_waiting{model_name="llama-8b"} 37
vllm:gpu_cache_usage_perc{model_name="llama-8b"} 0.97
```

Here 64 are running and 37 are waiting, with the cache 97 percent full. Users in the queue see high TTFT. More capacity is needed, or shorter contexts.

Scaling on the right signal

GPU use percentages are a poor signal. A GPU can read 100 percent "utilized" while only partly busy in terms of throughput, and CPU on the pod says nothing. Better signals:

- **Queue length** (requests waiting)
- **Concurrent requests** per replica compared with a tested maximum
- **KV cache usage**
- **TTFT or latency percentiles** as a guard

With KEDA or the Horizontal Pod Autoscaler on custom metrics, add replicas when the waiting count stays above a small number for a minute. Scale down slowly: a new replica needs minutes to schedule a GPU node, pull an image and load weights. Keep a minimum of one or two warm replicas for latency, and consider scaling a batch workload to zero.

Cost

Cost per token = GPU price per hour ÷ tokens generated per hour.

If a GPU costs 4 USD per hour and serves 2 500 tokens per second at full load, it produces 9 million tokens per hour, so about 0.44 USD per million tokens. If it only runs at 25 percent of that, the cost per million

MARGA by EknathaLabs marga.eknathalabs.com

tokens is four times higher, about 1.78 USD. **Utilisation** is the biggest lever. The ways to improve it are:

- Larger batches and continuous batching
- Quantization, so more fits and faster decode
- Smaller models for easy tasks, with routing to bigger ones when needed
- Prefix caching for repeated prompts
- Spot or preemptible GPUs for batch work that can restart, with checkpoints
- Reserved capacity for the stable base load

Compare against a hosted API price per million tokens at your real volume. Hosted wins at low or spiky use; self-hosting wins at steady high use, or when privacy and control matter.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Scaling, latency and cost fits together

Part	What it does	What goes wrong
What users feel	TTFT (time to first token): how long before anything appears.	Comparing GPU cost with API cost at a single quiet hour.
Scaling on the right signal	GPU use percentages are a poor signal. A GPU can read 100 percent "utilized" while only partly busy in terms of throughput, and CPU on the pod says nothing.	Scaling down fast and paying start-up time again.
Cost	Cost per token = GPU price per hour ÷ tokens generated per hour.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What users feel to Scaling on the right signal / Scaling on the right signal to Cost

Incident: Scaling on the wrong metric

Incident: *A launch-day traffic spike produced timeouts although autoscaling was enabled and CPU was low.*

```
$ kubectl get hpa llm
NAME  REFERENCE        TARGETS   MINPODS  MAXPODS  REPLICAS
llm   Deployment/llm    12%/60%   2        10       2
$ curl -s llm-0:8000/metrics | grep num_requests_waiting
vllm:num_requests_waiting{model_name="llama-8b"} 211
```

The autoscaler used CPU, which stays low when the heavy work happens on the GPU, so it saw nothing wrong while 211 requests waited. They switched to scaling on waiting requests per replica, with a rule of 10, and kept four replicas warm for events. New replicas still took six minutes to start, so a model image with weights baked in and a node pool with a spare node cut that to two. The metric that you scale on must move when users suffer.

Common mistakes

- Scaling GPU pods on CPU
- Averaging latency and ignoring p95
- Scaling down fast and paying start-up time again
- Comparing GPU cost with API cost at a single quiet hour

Check yourself

What does TTFT measure?

Answer: The time before the first token appears

MARGA by EknathaLabs marga.eknathalabs.com

Which signal is a better scaling trigger for LLM servers than CPU?

Answer: Number of waiting requests

A 4 USD per hour GPU produces 9 million tokens per hour. Roughly what does one million tokens cost?

Answer: About 0.44 USD

What is the biggest lever on cost per token?

Answer: GPU utilisation

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Monitoring and debugging GPU workloads

15 minutes. Read nvidia-smi, use DCGM metrics, and diagnose out-of-memory errors, throttling, hung multi-GPU jobs and failed cards.

GPU problems look like application problems until you look at the GPU. A few tools and patterns find most of them.

What you will learn

- Reading nvidia-smi
- DCGM and the metrics worth alerting on
- The usual faults: out of memory, throttling, XID errors, hangs
- A debugging order

nvidia-smi

```
$ nvidia-smi
| GPU  Name              Persistence-M | Bus-Id       Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf       Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|   0  NVIDIA A100-SXM4-80GB        Off | 00000000:07:00.0 Off |                    0 |
| N/A   71C    P0     392W / 400W |  78410MiB / 81920MiB |     99%      Default |
| Processes:
|  GPU  PID    Type  Process name                                GPU Memory Usage |
|    0  18211  C     /usr/bin/python3                                 78120MiB |
```

- **Memory-Usage** is the first thing to check for out-of-memory problems. The **Processes** list says which program holds it.
- **GPU-Util** is the share of time at least one kernel was running. It is not a measure of how efficient the work is.
- **Temp** and **Pwr** near the cap can lead to throttling.
- **ECC errors** (the "Volatile Uncorr. ECC" column) above zero point to a failing memory.
- `nvidia-smi dmon` streams utilization, and `nvidia-smi -q` prints everything.

DCGM and metrics

In Kubernetes, the **DCGM exporter** publishes GPU metrics for Prometheus. Useful ones include GPU utilization, framebuffer memory used, temperature, power, SM clock and XID errors. Alert on:

- Memory used close to total for several minutes
- Temperature or power at limits with clocks dropping (throttling)
- Any XID error or growing ECC error count
- Utilization near zero on a pod that holds a GPU (an idle reservation that costs money)

Common faults

Out of memory (OOM). The message names the sizes:

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 2.00 GiB.
GPU 0 has a total capacity of 79.15 GiB of which 1.12 GiB is free.
```

Check which process holds the memory (another model on the same card is common), reduce the batch size or context length, lower `--gpu-memory-utilization`, quantize, or split across GPUs. A leftover process from a crashed job can hold memory: find it in `nvidia-smi` and stop it.

Throttling. The GPU lowers its clocks when hot or at a power limit, and speed drops without an error. Check temperature, fan or cooling, and clock values. A card that is slow only in the afternoon often points

MARGA by EknathaLabs marga.eknathalabs.com

to cooling.

XID errors. The driver writes them to the kernel log:

```
$ dmesg | grep -i xid
NVRM: Xid (PCI:0000:07:00): 79, pid=18211, GPU has fallen off the bus.
```

XID 79 means the GPU has fallen off the bus: usually a hardware, power or seating fault. Drain the node, move the work, and have the hardware checked. Other codes have their own meanings in NVIDIA's documentation; look each one up.

Hung multi-GPU jobs. Training or tensor-parallel serving uses NCCL for communication. If one rank dies or the network between nodes fails, the others wait forever and show 100 percent utilization while doing nothing. Set `NCCL_DEBUG=INFO` to see where it stops, set timeouts, and check that all ranks started and the network interfaces are right.

Driver and CUDA mismatches. A container's CUDA version must be supported by the host driver. Errors such as "CUDA driver version is insufficient for CUDA runtime version" mean the driver needs updating or the image needs an older CUDA.

Debugging order

1. Is the GPU visible? `nvidia-smi`, then inside the container.
2. Is memory full, and who holds it?
3. Are temperature, power and clocks normal?
4. Does the kernel log show XID errors?
5. For multi-GPU work, did every rank start and can they talk?
6. Do driver and library versions match?

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Monitoring and debugging GPU workloads fits together

Part	What it does	What goes wrong
nvidia-smi	Memory-Usage is the first thing to check for out-of-memory problems.	Using GPU-Util as a measure of useful work.
DCGM and metrics	In Kubernetes, the DCGM exporter publishes GPU metrics for Prometheus.	Ignoring the kernel log for XID errors.
Common faults	Out of memory (OOM).	Running two memory-hungry processes on one card.
Debugging order	Is the GPU visible?. nvidia-smi, then inside the container.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: nvidia-smi to DCGM and metrics / DCGM and metrics to Common faults / Common faults to Debugging order

Incident: The job that ran at 100 percent and did nothing

Incident: *A four-GPU training job showed full utilization for six hours and no progress.*

```
$ kubectl logs trainer-2 | tail -2
[rank2] NCCL WARN Net : Connect to 10.0.4.17<45214> failed : Connection timed out
$ kubectl get pods -l job=train -o wide | grep -c Running
3
```

Only three of four workers were running. The fourth had been evicted from its node, and the other three waited for it inside a collective operation, which keeps the GPU shown as busy. Utilization was a misleading signal. The team added a job-level timeout so the group failed fast, restarts from checkpoints, and alerts when the number of running workers is below the expected count. A busy GPU is not the same

as progress.

Common mistakes

- Using GPU-Util as a measure of useful work
- Ignoring the kernel log for XID errors
- Running two memory-hungry processes on one card
- No timeouts on collective operations

Check yourself

Which tool shows GPU memory use and which process holds it?

Answer: nvidia-smi

What does XID 79, GPU has fallen off the bus, usually indicate?

Answer: A hardware, power or seating problem

Why can a hung multi-GPU job show 100 percent utilization?

Answer: GPUs wait inside a collective operation for a missing worker

What is a first step for CUDA out of memory?

Answer: See which process holds the memory and reduce batch or context

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Quantization and sizing a model for a GPU

12 minutes. Estimate GPU memory for weights and cache, understand quantization formats and choose a precision that fits your quality and cost needs.

Before choosing a GPU, you need to know how much memory the model will take. Most surprises come from forgetting the cache and overhead. Quantization is the main lever for fitting larger models into smaller, cheaper hardware.

What you will learn

- Calculating memory for weights
- Why the KV cache and overhead matter
- Common quantization formats and trade-offs
- Testing quality before you commit

The memory math

```
def weights_gb(params_billion, bytes_per_param):
    return params_billion * bytes_per_param

for name, b in [("fp16", 2.0), ("int8", 1.0), ("int4", 0.5)]:
    print(name, weights_gb(70, b), "GB for a 70B model")
```

Weights for a 70B model are about 140 GB in FP16, 70 GB in INT8 and 35 GB at 4 bits. Add the **KV cache**, which grows with context length and concurrent requests, plus activations and runtime overhead. A rule of thumb is to keep 10 to 20 percent of GPU memory free and to budget the cache explicitly.

Formats

- **FP16 or BF16:** the usual baseline, best quality, largest size.
- **FP8 and INT8:** about half the memory with small quality loss on modern hardware.
- **4-bit (AWQ, GPTQ, GGUF variants):** large savings, more quality risk, especially for small models and long reasoning.

Lower precision also reduces memory traffic, which often improves token rate, because decoding is limited by memory bandwidth.

Test before you ship

```
# serve a quantized model and compare with the baseline on your own prompts
vllm serve ./llama-70b-awq --quantization awq --max-model-len 8192 --gpu-memory-utilization 0.90
python eval_prompts.py --endpoint http://localhost:8000/v1 --set support-tickets.jsonl
```

Use a task-specific evaluation set, not only public benchmarks. Compare accuracy, refusal behavior and formatting, and measure latency and throughput. Record the chosen format with the model version so everyone serves the same thing.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Quantization and sizing a model for a GPU fits together

Part	What it does	What goes wrong
The memory math	Weights for a 70B model are about 140 GB in FP16, 70 GB in INT8 and 35 GB at 4 bits.	Counting weights but not the KV cache.
Formats	FP16 or BF16: the usual baseline, best quality, largest size.	Filling GPU memory to 100 percent.
Test before you ship	Use a task-specific evaluation set, not only public benchmarks.	Choosing a format using only public benchmarks.

Flow: The memory math to Formats / Formats to Test before you ship

Incident: Out of memory on load

Incident: *A 70B model in FP16 could not load on one 80 GB GPU.*

```
torch.cuda.OutOfMemoryError: CUDA out of memory.
Tried to allocate 1.88 GiB
GPU 0 capacity 79.2 GiB, 1.1 GiB free
```

The team had counted only parameters and assumed it would fit. In FP16 the weights alone needed about 140 GB. They moved to a 4-bit format, reserved cache memory in the plan and verified quality on their ticket dataset before rollout.

Common mistakes

- Counting weights but not the KV cache
- Choosing a format using only public benchmarks
- Filling GPU memory to 100 percent
- Not recording which quantization is deployed

Check yourself

How much does a 70B model need in 4-bit weights?

Answer: About 35 GB

What grows with context length and concurrency?

Answer: The KV cache

Why can lower precision be faster?

Answer: Less memory traffic

What should drive format choice?

Answer: Quality on your own tasks

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Batching, KV cache and throughput

13 minutes. Understand continuous batching, prefill and decode, KV cache paging and the knobs that trade latency for throughput.

Serving a language model is not one calculation per request. Each user needs a long series of small steps, and the GPU wastes most of its power if it does them alone. Batching and cache management are what make serving affordable.

What you will learn

- Prefill versus decode and why they behave differently
- Continuous batching and paged KV cache
- The main limits and what they trade
- Reading throughput and latency metrics

Two phases

Prefill reads the whole prompt in one parallel pass and is compute heavy. It determines the **time to first token**. **Decode** produces one token per step, reuses the KV cache and is limited by memory bandwidth. It determines **inter-token latency**.

Continuous batching

Static batching waits for a full batch and runs until the longest request ends. **Continuous batching** adds new requests and removes finished ones at every step, so the GPU stays busy. **Paged attention** stores the KV cache in small blocks, reducing waste and allowing more concurrent sequences.

```
vllm serve meta/model-8b \
  --max-num-seqs 128 \
  --max-num-batched-tokens 16384 \
  --max-model-len 8192 \
  --gpu-memory-utilization 0.90
```

The knobs

- **max-num-seqs:** more concurrent requests means higher throughput but more cache use and higher latency per request.
- **max-num-batched-tokens:** large values favor throughput, small ones favor smooth latency.
- **max-model-len:** a smaller limit leaves memory for more sequences.
- **Prefix caching:** reuses cache for shared prompt starts, such as a system prompt.

Measure

```
curl -s localhost:8000/metrics | grep -E "num_requests_(running|waiting)|time_to_first_token|gpu_c \
    ache_usage"
```

Watch queue length, cache usage, time to first token and tokens per second under realistic concurrency. Rising waiting requests with high cache usage means you need more memory, smaller limits or more replicas.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Batching, KV cache and throughput fits together

Part	What it does	What goes wrong
Two phases	Prefill reads the whole prompt in one parallel pass and is compute heavy.	Ignoring queue depth and cache usage.
Continuous batching	Static batching waits for a full batch and runs until the longest request ends.	Allowing very long contexts for every route.
The knobs	max-num-seqs: more concurrent requests means higher throughput but more cache use and higher latency per request.	Tuning with a single test request.
Measure	Watch queue length, cache usage, time to first token and tokens per second under realistic concurrency.	Setting memory utilization to 100 percent.

Flow: Two phases to Continuous batching / Continuous batching to The knobs / The knobs to Measure

Incident: Fast alone, slow together

Incident: *Latency jumped when concurrency increased because the cache was full.*

```
num_requests_running: 64
num_requests_waiting: 211
gpu_cache_usage_perc: 0.99
time_to_first_token p95: 9.4s
```

The server accepted long contexts from every request and exhausted cache memory, so new requests waited. The team capped context length for the public route, enabled prefix caching for the shared system prompt and added a replica with autoscaling on queue depth.

Common mistakes

- Tuning with a single test request
- Allowing very long contexts for every route
- Ignoring queue depth and cache usage
- Setting memory utilization to 100 percent

Check yourself

Which phase decides time to first token?

Answer: Prefill

What does continuous batching do?

Answer: Adds and removes requests every step

What does high cache usage with waiting requests suggest?

Answer: Memory limits concurrency

What does prefix caching reuse?

Answer: Cache of a shared prompt start

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Multi-GPU serving: tensor and pipeline parallelism

12 minutes. Split a model across GPUs with tensor and pipeline parallelism, understand interconnect needs and decide when replicas are better.

Large models need more memory than one GPU has. Splitting them works, but every split adds communication, so how and where you split changes speed dramatically.

What you will learn

- Tensor versus pipeline parallelism
- Why interconnect bandwidth matters
- Reading the GPU topology
- When to scale with replicas instead

Two ways to split

- **Tensor parallelism (TP):** every layer's weights are divided across GPUs, and GPUs exchange results at each layer. It reduces latency for a single request but needs very fast links.
- **Pipeline parallelism (PP):** each GPU holds a block of layers, and activations flow from stage to stage. Communication is lighter, but stages can sit idle without enough concurrent requests.

```
nvidia-smi topo -m
vllm serve meta/model-70b \
  --tensor-parallel-size 4 \
  --max-model-len 8192
```

Topology

`nvidia-smi topo -m` shows how GPUs connect. **NV#** entries mean NVLink, which is fast. **PIX** or **PHB** mean paths through PCIe, which is slower. **SYS** crosses CPU sockets and is slowest. Choose GPUs that share fast links for the tensor-parallel group, and keep the group inside one node when possible.

Replicas versus splitting

If a model fits on one GPU, run several independent replicas behind a load balancer. This gives near-linear throughput, no communication overhead and isolated failures. Split only when memory requires it, and then replicate the split group.

```
kubectl get pods -l app=llm -o wide
kubectl describe node gpu-node-3 | grep -A5 "nvidia.com/gpu"
```

Check that all pods of one tensor-parallel group are scheduled on the same node, using node affinity or a multi-GPU resource request.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Multi-GPU serving: tensor and pipeline parallelism fits together

Part	What it does	What goes wrong
Two ways to split	Tensor parallelism (TP): every layer's weights are divided across GPUs, and GPUs exchange results at each layer.	Tensor parallelism across slow links.
Topology	nvidia-smi topo -m shows how GPUs connect.	Ignoring topology when placing pods.
Replicas versus splitting	If a model fits on one GPU, run several independent replicas behind a load balancer.	Splitting a model that fits on one GPU.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Two ways to split to Topology / Topology to Replicas versus splitting

Incident: Slower with more GPUs

Incident: *Adding GPUs across nodes made serving slower.*

```
NCCL INFO Channel 00 via NET/Socket
tokens/s: 21 (single GPU: 34)
allreduce latency: high
```

The tensor-parallel group spanned two machines connected by regular Ethernet, so each layer waited on the network. The team kept tensor parallelism within a node with NVLink, used replicas to scale out and added a scheduling rule that keeps each group on one node.

Common mistakes

- Tensor parallelism across slow links
- Splitting a model that fits on one GPU
- Ignoring topology when placing pods
- Using an odd GPU count that does not divide the model heads

Check yourself

When use replicas instead of splitting?

Answer: When the model fits on one GPU

What does NVLink provide?

Answer: Fast GPU-to-GPU links

Which command shows the GPU topology?

Answer: nvidia-smi topo -m

Where should a tensor-parallel group live?

Answer: Inside one node with fast links

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Autoscaling LLM inference

13 minutes. Scale GPU inference on queue depth and latency instead of CPU, handle slow startup with warm capacity and control cost.

GPU nodes are expensive and slow to start, so autoscaling inference is harder than scaling a web service. The right signal, warm capacity and sensible limits make the difference between smooth scaling and timeouts.

What you will learn

- Choosing a scaling signal for model servers
- Why cold start dominates scale-up time
- Configuring KEDA or an HPA on custom metrics
- Controlling cost with limits and schedules

The signal

Scale on what users feel. Good signals are **queue depth**, **running requests per replica** and **time to first token**. GPU utilization alone can look high while a replica is perfectly healthy, and CPU says almost nothing.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: llm-scaler
spec:
  scaleTargetRef:
    name: llm-server
  minReplicaCount: 2
  maxReplicaCount: 12
  cooldownPeriod: 600
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring:9090
        query: sum(vllm:num_requests_waiting)
        threshold: "8"
```

Cold start

Adding capacity needs a new GPU node, then an image pull, then a model download and weight load. That can take minutes. Reduce it with pre-pulled images, models on a fast local or shared volume, a pool of warm spare replicas and cluster autoscaler or Karpenter settings that keep GPU nodes ready.

Cost control

Set a **maximum replica count** and a budget alert. Use a long scale-down delay so replicas are not removed during short dips, because reloading is costly. For predictable peaks, scale up on a schedule before the peak:

```
kubectl get scaledobject,hpa -n ai
kubectl get nodes -l node.kubernetes.io/instance-type=gpu -o wide
```

Review the cost per thousand requests monthly.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Autoscaling LLM inference fits together

Part	What it does	What goes wrong
The signal	Scale on what users feel. Good signals are queue depth, running requests per replica and time to first token.	Aggressive scale-down that causes reloads.
Cold start	Adding capacity needs a new GPU node, then an image pull, then a model download and weight load.	No warm capacity for slow-starting models.
Cost control	Set a maximum replica count and a budget alert.	No maximum replicas or budget alert.

Flow: The signal to Cold start / Cold start to Cost control

Incident: Ten minutes of timeouts

Incident: *A traffic spike outran GPU node startup.*

```
09:00 queue depth 340
09:01 scale up 2 -> 8
09:09 replicas ready
429 rate 41% until then
```

Scaling worked, but new replicas needed minutes to start. The team kept a warm pool, moved model files to a local cache, added a scheduled scale-up before the known morning peak and rate-limited low-priority traffic during spikes.

Common mistakes

- Scaling on CPU
- No warm capacity for slow-starting models
- Aggressive scale-down that causes reloads
- No maximum replicas or budget alert

Check yourself

Which signal reflects inference load?

Answer: Queue depth or time to first token

What dominates scale-up time?

Answer: Node start and model loading

Why keep warm replicas?

Answer: To absorb spikes while new ones start

How handle predictable peaks?

Answer: Scheduled scale-up before them

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Model loading, caching and cold starts

12 minutes. Load large model files quickly with local caches and fast storage, use safetensors and shared volumes and plan rollouts around start-up time.

A model server cannot serve until its weights are in GPU memory. Getting there involves pulling a large image, fetching files and moving them across the bus, and each step offers a chance to speed up or to fail.

What you will learn

- Where start-up time goes
- Caching weights and images close to the GPU
- Readiness and startup probes for model servers
- Rolling updates that keep capacity

Where time goes

1. Pull the container image, often many gigabytes with CUDA libraries.
2. Download or read the model weights.
3. Load weights to GPU memory and initialize kernels.
4. Warm up with a few requests.

Measure each step with timestamps so you know which to fix.

Caching

```
volumes:
  - name: models
    persistentVolumeClaim:
      claimName: model-cache      # fast shared or local NVMe volume
containers:
  - name: server
    image: registry.example.com/llm-server:1.8
    args: ["--model", "/models/llama-8b", "--load-format", "safetensors"]
    volumeMounts:
      - {name: models, mountPath: /models, readOnly: true}
```

Use **safetensors**, which load quickly and avoid unsafe code execution. Populate the volume once with a job, verify file checksums and mount it read-only. Pre-pull images onto GPU nodes, or use an image cache in the cluster.

Probes and rollouts

```
startupProbe:
  httpGet: {path: /health, port: 8000}
  failureThreshold: 60
  periodSeconds: 10
readinessProbe:
  httpGet: {path: /v1/models, port: 8000}
```

The startup probe allows a long load without being killed, and readiness keeps traffic away until the model answers. Roll out with `maxUnavailable: 0` and a small `maxSurge`, so old replicas keep serving until new ones are ready. Remember that a surge needs spare GPUs.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Model loading, caching and cold starts fits together

Part	What it does	What goes wrong
Where time goes	Pull the container image, often many gigabytes with CUDA libraries.	Writable model volumes shared by many pods.
Caching	Use safetensors, which load quickly and avoid unsafe code execution.	Liveness probes that fire during model loading.
Probes and rollouts	The startup probe allows a long load without being killed, and readiness keeps traffic away until the model answers.	Restarting all replicas together.

Flow: Where time goes to Caching / Caching to Probes and rollouts

Incident: Killed while still loading

Incident: *Pods restarted repeatedly because probes were too strict for a slow load.*

```
Liveness probe failed: connection refused
Container server failed liveness probe, will be restarted
Back-off restarting failed container (CrashLoopBackOff)
```

The liveness probe started after 30 seconds, but loading took four minutes, so Kubernetes kept killing the pod just before it was ready. A startup probe with a long allowance and a readiness probe on the model endpoint fixed the loop.

Common mistakes

- Liveness probes that fire during model loading
- Downloading weights from the internet on every start
- Restarting all replicas together
- Writable model volumes shared by many pods

Check yourself

Which probe handles slow startup?

Answer: Startup probe

Why use safetensors?

Answer: Fast, safe weight loading

How keep capacity during updates?

Answer: maxUnavailable 0 with a surge

Why mount weights read-only?

Answer: Prevent accidental changes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. GPU sharing: MIG, time-slicing and MPS

12 minutes. Share one GPU among several workloads with MIG partitions, time-slicing or MPS and understand isolation, performance and fit.

Giving each small job a whole GPU wastes money. Sharing is possible in several ways, and they differ mostly in how well jobs are isolated from each other.

What you will learn

- How MIG partitions a GPU
- How time-slicing and MPS differ
- Requesting shared GPUs in Kubernetes
- Choosing a method by workload

MIG

Multi-Instance GPU on supported data center GPUs splits a card into up to seven instances, each with its own memory, cache and compute. A fault or heavy use in one instance does not affect others.

```
nvidia-smi -i 0 -mig 1
nvidia-smi mig -cgi 1g.10gb,1g.10gb,2g.20gb -C
nvidia-smi -L
```

In Kubernetes the GPU Operator advertises slices as resources such as `nvidia.com/mig-1g.10gb`.

```
resources:
  limits:
    nvidia.com/mig-1g.10gb: 1
```

Time-slicing and MPS

Time-slicing makes several pods share a GPU by taking turns. It works on any GPU but gives no memory isolation, so one pod can exhaust memory and others fail, and latency is unpredictable.

```
# device plugin config: advertise each GPU as 4 shares
sharing:
  timeSlicing:
    resources:
      - name: nvidia.com/gpu
        replicas: 4
```

MPS lets processes run kernels at the same time and can raise utilization for many small workloads from one user, with limited isolation.

Choosing

Use **MIG** for multi-tenant or latency-sensitive inference. Use **time-slicing** for development, notebooks and tests where interference is acceptable. Use **whole GPUs** for large models and training. Monitor per-slice memory and utilization, and set limits so that one team cannot take all slices.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How GPU sharing: MIG, time-slicing and MPS fits together

Part	What it does	What goes wrong
MIG	Multi-Instance GPU on supported data center GPUs splits a card into up to seven instances, each with its own memory, cache and compute.	Forgetting that time-slicing has no memory isolation.
Time-slicing and MPS	Time-slicing makes several pods share a GPU by taking turns.	Using time-slicing for production inference.
Choosing	Use MIG for multi-tenant or latency-sensitive inference.	Creating MIG layouts that waste slices.

Flow: MIG to Time-slicing and MPS / Time-slicing and MPS to Choosing

Incident: One notebook starved the rest

Incident: *A shared GPU ran out of memory because one pod took almost all of it.*

```
CUDA error: out of memory
nvidia-smi: 79 GiB used by pid 4412
3 neighbouring pods failed to start
```

Time-slicing gave four pods one card but no memory limits, and one notebook loaded a huge tensor. The team moved shared workloads to MIG slices with fixed memory and kept time-slicing only for short-lived development sessions with quotas.

Common mistakes

- Using time-slicing for production inference
- Forgetting that time-slicing has no memory isolation
- Creating MIG layouts that waste slices
- No quotas on shared GPUs

Check yourself

What does MIG give?

Answer: Hardware-isolated GPU partitions

Does time-slicing isolate memory?

Answer: No

Which suits multi-tenant inference?

Answer: MIG

Where is a MIG resource requested?

Answer: In container resource limits

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. GPU failures, Xid errors and recovery

12 minutes. Read GPU error codes, tell memory, thermal, link and driver faults apart, and drain, reset and replace GPUs safely.

GPU hardware is dense, hot and heavily used, so failures are normal. The kernel and NVIDIA tools report what went wrong, if you know where to look and what to do next.

What you will learn

- Reading Xid messages and ECC counters
- Thermal, power and link problems
- Draining a node and resetting or replacing a GPU
- Automating detection and isolation

Where to look

```
dmesg -T | grep -i "NVRM: Xid"
nvidia-smi -q -d ECC | grep -E "Uncorrectable|Correctable" | head
nvidia-smi --query-gpu=index,temperature.gpu,power.draw,clocks_throttle_reasons.active --format=cs \
    v
dcgmi diag -r 2
```

An **Xid** is an error code from the driver. Some point to an application bug, such as an illegal memory access, but others point to hardware: uncorrectable memory errors, a GPU that fell off the bus or repeated link errors. Look up the code in NVIDIA's table before acting.

Fault families

- **Memory:** rising uncorrectable ECC errors or row remapping failures mean the GPU should be retired.
- **Thermal and power:** throttling reasons, high temperature and power capping point to cooling or power limits in the rack.
- **Link:** NVLink or PCIe errors can come from a seating problem, a cable or a failing board.
- **Driver and software:** hangs after upgrades or container mismatches are fixed by matching driver and CUDA versions.

Respond

```
kubectl cordon gpu-node-7
kubectl drain gpu-node-7 --ignore-daemonsets --delete-emptydir-data
nvidia-smi --gpu-reset -i 3      # only if no process uses the GPU
```

Cordon and drain before touching hardware. A reset can clear transient faults, but repeated Xid errors or ECC failures mean replacement. Record the serial number and evidence for the vendor.

Automate

Run DCGM health checks as a daemon set, export metrics to Prometheus and alert on Xid counts. A controller can taint a failing node automatically so new jobs avoid it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How GPU failures, Xid errors and recovery fits together

Part	What it does	What goes wrong
Where to look	An Xid is an error code from the driver.	Touching hardware before draining the node.
Fault families	Memory: rising uncorrectable ECC errors or row remapping failures mean the GPU should be retired.	Treating every Xid as a software bug.
Respond	Cordon and drain before touching hardware.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Automate	Run DCGM health checks as a daemon set, export metrics to Prometheus and alert on Xid counts.	No alert on Xid or ECC counters.

Flow: Where to look to Fault families / Fault families to Respond / Respond to Automate

Incident: A GPU that fell off the bus

Incident: *A node lost one GPU in the middle of a job.*

```
NVRM: Xid (PCI:0000:3b:00): 79, GPU has fallen off the bus.
nvidia-smi: Unable to determine the device handle for GPU 3
job failed at step 18402
```

Xid 79 meant the card disappeared from the PCIe bus. A reboot brought it back briefly, but the error returned under load. The team drained the node, reseated and then replaced the GPU, and added an automatic taint on Xid 79 and 48 so jobs no longer landed there.

Common mistakes

- Treating every Xid as a software bug
- Resetting repeatedly instead of replacing
- Touching hardware before draining the node
- No alert on Xid or ECC counters

Check yourself

What is an Xid?

Answer: A driver error code

What do rising uncorrectable ECC errors suggest?

Answer: A failing GPU memory

What should you do before servicing a GPU node?

Answer: Cordon and drain it

How automate isolation?

Answer: Taint nodes on bad health signals

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: GPU, AI infrastructure and LLM serving

Memory sums, nvidia-smi, vLLM flags, Kubernetes GPU scheduling and the metrics that explain slow or failing inference.

Memory sum

Item	Rule of thumb
Weights, 16-bit	parameters x 2 bytes (7B is about 14 GB)
Weights, 8-bit	parameters x 1 byte
Weights, 4-bit	parameters x 0.5 byte
KV cache	grows with context length x concurrent requests
Overhead	leave 10 to 20 percent free

Look at the GPU

```
nvidia-smi
nvidia-smi --query-gpu=name,memory.used,memory.total,utilization.gpu,temperature.gpu --format=csv
nvidia-smi dmon -c 5
nvidia-smi topo -m
```

> **Warning:** *High memory use with low utilisation usually means the GPU is waiting for the CPU or the network, not that it is busy.*

Serve with vLLM

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
  --max-model-len 8192 \
  --gpu-memory-utilization 0.90 \
  --tensor-parallel-size 1
curl localhost:8000/v1/models
curl localhost:8000/metrics | grep -E "num_requests|cache_usage"
```

Flag	Effect
`--max-model-len`	longest context, drives KV cache size
`--gpu-memory-utilization`	share of GPU memory the engine may use
`--tensor-parallel-size`	split the model over N GPUs

Kubernetes

```
resources:
  limits:
    nvidia.com/gpu: 1
tolerations:
- key: nvidia.com/gpu
  operator: Exists
  effect: NoSchedule

kubectl describe node gpu-1 | grep -A3 "nvidia.com/gpu"
kubectl get pods -A -o wide | grep -i nvidia
kubectl describe pod llm-0 | tail -15
```

MARGA by EknathaLabs marga.eknathalabs.com

Signals to watch

Metric	Meaning
time to first token	how long users wait to see output
tokens per second	generation speed
requests waiting	demand above capacity
KV cache usage	near 100 percent means queueing and evictions
GPU memory and utilisation	fit and efficiency
Xid errors in dmesg	hardware or driver faults

Quick GPU checks

```
nvidia-smi --query-gpu=name,memory.used,memory.total,utilization.gpu,temperature.gpu --format=csv
nvidia-smi dmon -c 3
kubectl describe node gpu-1 | grep -A3 nvidia.com/gpu
kubectl logs deploy/llm --tail=50
```

Test a server

```
curl -s localhost:8000/v1/models
curl -s localhost:8000/v1/chat/completions -H "Content-Type: application/json" -d '{"model":"m","m \
    essages":[{"role":"user","content":"hi"}],"max_tokens":16}'
curl -s localhost:8000/metrics | grep num_requests
```

Cautions

Danger: *Setting* `--gpu-memory-utilization` *near 1.0 leaves no room for spikes and invites out-of-memory crashes.*

Warning: `nvidia-smi --gpu-reset` *and driver changes interrupt every process on the card. Drain the node first.*

Warning: *Autoscaling a model server on CPU use does not work, as the work happens on the GPU. Use queue depth or latency.*

Caution: *A long context grows the KV cache for every request. Set a maximum that fits your memory.*

MARGA by EknathaLabs
marga.eknathalabs.com